

Day 4 Learning

#Wrapper Classes in java:

- Java is object-oriented but not purely so because it contains primitive data types (e.g., int, float, double) which are not objects.
- Primitive types like `int a = 1;` do not have methods or properties, unlike objects such as `String`.
- For example, `a.` shows no accessible methods, while `String str = "Vipul"; str.` shows many methods.
- Primitive types are stored in the stack memory, hold simple values, and cannot be used where objects are required, such as in generic collections (`List<int>` results in an error).
- To use primitives where objects are required (e.g., in collections), Java provides wrapper classes.

Primitive Type	Corresponding Wrapper Class
int	Integer
float	Float
char	Character
boolean	Boolean
double	Double
long	Long
byte	Byte
short	Short

- Wrapper classes “wrap” primitive values into objects stored on the heap, allowing method calls and usage in collections.
- Example:

```
int a = 1;    // primitive variable, no methods
```

```
Integer b = 1; // object, methods accessible e.g. b.hashCode()
```

Java supports autoboxing and unboxing:

- Autoboxing: automatic conversion of primitive to wrapper object (int → Integer).
- Unboxing: automatic conversion of wrapper object to primitive (Integer → int).

This feature simplifies code, avoiding verbose manual conversions like Integer.valueOf(a) or integerObj.intValue().

Wrapper classes provide many useful static and instance methods, e.g.,

- Integer.MAX_VALUE
- Integer.min(a, b)
- Integer.toString(10) converts number to binary string.

Primitive variables cannot hold null, but wrapper objects can, which is useful in certain scenarios.

Key Concepts:

- Java uses primitive types for efficiency but supports wrapper classes to treat primitives as objects when needed.
-
- Autoboxing/unboxing simplifies conversions between primitives and wrapper classes.
-
- Understanding references and memory allocation is crucial in Java to avoid confusion about object manipulation and passing objects to functions.
-
- Equality checks differ between primitives and objects, requiring .equals() for object content comparison.
-
- Java's design balances performance (primitive types) with full object-oriented features (wrapper classes).

#Exception Handling in Java:

An Exception is an unwanted or unexpected event that occurs during the execution of a program, disrupting the normal flow of instructions. Distinction between Errors and Exceptions:

- Errors: These represent critical conditions that an application should not try to catch, such as `OutOfMemoryError` or `StackOverflowError`. These are usually related to the environment or the JVM.
- Exceptions: These are conditions that a reasonable application might want to catch and handle, such as `FileNotFoundException` or `ArithmeticException`.

Exception Hierarchy and Types (Checked and Unchecked):

In Java, all exception and error classes are subclasses of the `Throwable` class.

- Checked Exceptions: These are checked at compile-time. The compiler forces the programmer to either handle them using a try-catch block or declare them using the `throws` keyword. Examples: `IOException`, `SQLException`.
- Unchecked Exceptions: Also known as Runtime Exceptions. These are not checked at compile-time and usually occur due to programming logic flaws. Examples: `NullPointerException`, `ArrayIndexOutOfBoundsException`.

Exception Handling Keywords (try, catch, finally, throw, throws)

- `try`: This block contains the code that might throw an exception.
- `catch`: This block is used to handle the exception thrown by the `try` block.
- `finally`: This block always executes regardless of whether an exception is handled or not. It is primarily used for resource cleanup.
- `throw`: This keyword is used to explicitly throw a single exception from a method or block of code.
- `throws`: This keyword is used in the method signature to indicate that the method might throw one or more exceptions.

try-catch-finally and Multiple Catch Blocks

A single `try` block can be followed by multiple catch blocks. This is useful when a code block can generate different types of exceptions. Rule: Always place specific exception catch blocks before the general `Exception` class catch block to avoid compile-time errors.

Throwable Class Methods and Advanced Concepts

Key methods of the `Throwable` class:

- `getMessage()`: Returns a detailed message about the exception.

- `printStackTrace()`: Prints the exception name, message, and the line number where it occurred.
- `toString()`: Returns a short description of the throwable.

Advanced Concepts:

- **Multi-catch:** Introduced in Java 7, it allows handling multiple exceptions in a single catch block using the pipe (`|`) operator.
- **Chaining:** This allows you to associate one exception with another, usually using the `initCause()` method to show the root cause.
- **Suppressed Exceptions:** When using try-with-resources, if multiple exceptions occur, the primary exception is thrown and others are stored as suppressed.

Exception Propagation

If an exception is not caught, it moves up the call stack to the calling method. This process continues until a handler is found.

- Unchecked exceptions propagate automatically.
- Checked exceptions require the `throws` keyword to propagate. In Constructors, exceptions can be handled, but if a constructor throws an exception, the object creation fails.

Custom Exceptions and Best Practices

Developers can create their own exceptions by extending the `Exception` class (for checked) or `RuntimeException` class (for unchecked). Rules and Best Practices:

- Catch specific exceptions instead of just the generic `Exception` class.
- Always close resources using `finally` or `try-with-resources`.
- Never leave a catch block empty (swallowing exceptions).
- Use custom exceptions for specific business logic failures.

try-with-resources and Special Comparisons

try-with-resources: A Java 7 feature that ensures resources like files or database connections are closed automatically. The resource must implement the `AutoCloseable` interface.

Comparison:

- **final:** A keyword used to make variables constant, methods non-overridable, or classes non-inheritable.

- **finally:** A block used in exception handling to execute important code (like cleanup) after try-catch.
- **finalize():** A method in the Object class called by the Garbage Collector before an object is removed from memory (now deprecated).

#Checked and Unchecked exceptions in java:

1. Checked Exceptions (Compile-time Exceptions)

These are exceptions that the Java compiler requires you to handle (using **try-catch**) or declare (using **throws**). They usually represent conditions outside the control of the program (e.g., file system or network issues).

- **IOException:** Thrown when an I/O operation (like reading a file) fails or is interrupted.
- **FileNotFoundException:** A subtype of **IOException**; occurs when a file at a specified pathname does not exist.
- **SQLException:** Occurs when there is an error related to database access or connectivity.
- **ClassNotFoundException:** Thrown when an application tries to load a class through its string name but no definition for the class can be found.
- **InterruptedException:** Thrown when a thread that is sleeping, waiting, or otherwise occupied is interrupted.
- **NoSuchMethodException:** Thrown when a particular method cannot be found.
- **InstantiationException:** Thrown when an application tries to create an instance of a class using **newInstance** but the specified class object cannot be instantiated (e.g., it is an abstract class).

2. Unchecked Exceptions (Runtime Exceptions)

These are exceptions that occur at runtime. The compiler does not force you to handle them because they are usually the result of programming errors or logic flaws.

- **NullPointerException (NPE):** The most famous one; thrown when you attempt to use an object reference that has a **null** value.
- **ArrayIndexOutOfBoundsException:** Thrown when you try to access an array index that is either negative or greater than or equal to the size of the array.
- **ArithmeticException:** Occurs when an exceptional arithmetic condition has happened (e.g., dividing an integer by zero).

- **IllegalArgumentException:** Thrown to indicate that a method has been passed an illegal or inappropriate argument.
- **NumberFormatException:** A subtype of `IllegalArgumentException`; occurs when you try to convert a string into a numeric type, but the string does not have the appropriate format.
- **ClassCastException:** Thrown when you try to cast an object to a subclass of which it is not an instance.
- **IndexOutOfBoundsException:** A general version of the array exception; often seen with `ArrayList` or `String` operations.

```
java.lang.Object
├── java.lang.Throwable
│   ├── java.lang.Error
│   │   ├── java.lang.VirtualMachineError
│   │   ├── java.lang.OutOfMemoryError
│   │   ├── java.lang.StackOverflowError
│   │   └── java.lang.LinkageError
│   │       ├── java.lang.ClassFormatError
│   │       └── java.lang.NoClassDefFoundError
│   └── java.lang.Exception
│       ├── java.lang.RuntimeException
│       │   ├── java.lang.ArithmeticException
│       │   ├── java.lang.NullPointerException
│       │   ├── java.lang.IndexOutOfBoundsException
│       │   └── java.lang.IllegalArgumentException
│       ├── java.io.IOException
│       │   ├── java.io.FileNotFoundException
│       │   └── java.io.EOFException
│       ├── java.sql.SQLException
│       ├── java.lang.reflect.InvocationTargetException
│       ├── java.net.SocketException
│       └── java.net.UnknownHostException
```

